

Expected Result (ผลลัพธ์ที่ต้องได้)

1. GET ทั้งหมด

The screenshot shows a REST client interface with a GET request to `http://localhost:8080/coffees`. The response is displayed in JSON format, showing a list of two coffee objects.

Query Params

Key	Value
Key	Value

Body | Cookies | Headers (5) | Test Results

Pretty | Raw | Preview | Visualize | JSON |

```
1  [
2    {
3      "id": "6a281d90-afa3-41d7-9a39-a68c4308a1ec",
4      "name": "Espresso",
5      "price": 55.0
6    },
7    {
8      "id": "b6409fa8-7ecd-42be-b329-fa3ad0112ce7",
9      "name": "Latte",
10     "price": 50.0
11   }
12 ]
```

2. GET ตาม id

GET



http://localhost:8080/coffees/b8b986af-0db7-42ff-ac39-afeb9940edac

```
1  {  
2    "id": "b8b986af-0db7-42ff-ac39-afeb9940edac",  
3    "name": "Espresso",  
4    "price": 55.0  
5  }
```

3. POST เพิ่มเมนูใหม่

POST ▼ | http://localhost:8080/coffees

Params Authorization Headers (8) **Body** ● Pre-request Script Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary **JSON** ▼

```
1 { "name": "Cappuccino", "price": 60.0 }
```

```
1 {  
2   "id": "dd5cbb52-2519-4725-b38f-4c8d487cfd0e",  
3   "name": "Cappuccino",  
4   "price": 60.0  
5 }
```

```
1 {  
2   {  
3     "id": "6a281d90-afa3-41d7-9a39-a68c4308a1ec",  
4     "name": "Espresso",  
5     "price": 55.0  
6   },  
7   {  
8     "id": "b6409fa8-7ecd-42be-b329-fa3ad0112ce7",  
9     "name": "Latte",  
10    "price": 65.0  
11  },  
12  {  
13    "id": "dd5cbb52-2519-4725-b38f-4c8d487cfd0e",  
14    "name": "Cappuccino",  
15    "price": 60.0  
16  }  
17 }
```

4. PUT แก้วไข

PUT ▼ http://localhost:8080/coffees/b6409fa8-7ecd-42be-b329-fa3ad0112ce7

Params Authorization Headers (8) **Body** ● Pre-request Script Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary **JSON** ▼

1 `{"name": "Latte", "price": 50.0}`

```
1 {
2   "id": "b6409fa8-7ecd-42be-b329-fa3ad0112ce7",
3   "name": "Latte",
4   "price": 50.0
5 }
```

5. DELETE လာ

DELETE  http://localhost:8080/coffees/dd5cbb52-2519-4725-b38f-4c8d487cfd0e

Body

Cookies

Headers (5)

Test Results

Pretty

Raw

Preview

Visualize

Text 



1 Deleted coffee with ID: dd5cbb52-2519-4725-b38f-4c8d487cfd0e

GET  http://localhost:8080/coffees

Params

Authorization

Headers (6)

Body

Pre-request Script

Tests

Query Params

	Key	Value
	Key	Value

Body

Cookies

Headers (5)

Test Results

Pretty

Raw

Preview

Visualize

JSON 



1 [

2 {

3 "id": "6a281d90-afa3-41d7-9a39-a68c4308a1ec",

4 "name": "Espresso",

5 "price": 55.0

6 },

7 {

8 "id": "b6409fa8-7ecd-42be-b329-fa3ad0112ce7",

9 "name": "Latte",

10 "price": 50.0

11 }

12]

6. Error 404 (Not Found)

- หาไม่เจอเพราะยังไปหา coffee ควรเป็น coffees ทำให้ path ไม่ถูกต้อง

GET



http://localhost:8080/coffee

```
1 {  
2   "timestamp": "2026-07-22T09:05:37.870Z",  
3   "status": 404,  
4   "error": "Not Found",  
5   "path": "/coffee"  
6 }
```

คำตอบส่วน Discussion

1. HTTP method แต่ละตัว (GET/POST/PUT/DELETE) ต่างกันอย่างไร ยกตัวอย่างจากโปรเจกต์ตัวเอง

GET ใช้สำหรับดึงข้อมูล โดยไม่มีการแก้ไขข้อมูลในระบบ เช่น GET /coffees เพื่อดึงรายการเมนูกาแฟทั้งหมด

POST ใช้สำหรับสร้างข้อมูลใหม่ลงในระบบ เช่น POST /coffees โดยส่งชื่อและราคากาแฟใน Body เพื่อเพิ่มเมนูกาแฟใหม่

PUT ใช้สำหรับอัปเดต/แก้ไขข้อมูลเดิมที่มีอยู่แล้ว เช่น PUT /coffees/{id} เพื่อแก้ไขข้อมูลของกาแฟตาม ID ที่ระบุ

DELETE ใช้สำหรับลบข้อมูลออกจากระบบ เช่น DELETE /coffees/{id} เพื่อลบเมนูกาแฟออกจาก List ตาม ID ที่ระบุ

2. ทำไมต้องแยก Controller กับ Service ออกจากกัน มีข้อดีอย่างไรถ้าโปรแกรมโตขึ้น

Controller ทำหน้าที่รับ Request/Response ผัง HTTP เท่านั้น ส่วน Service จะดูแล Logic การทำงานของธุรกิจ ข้อดีเมื่อโปรแกรมโตขึ้นทำให้โค้ดเป็นระเบียบ อ่านง่าย ซ่อมบำรุงสะดวก สามารถนำ Logic ใน Service ไป reuse ใช้ร่วมกันในส่วนอื่นได้ง่าย และช่วยให้เขียน Unit Test ทดสอบระบบได้อย่างเป็นอิสระจากชั้น Web Server

3. ข้อมูลที่เก็บไว้ใน List ใน memory หายไปตอนไหน และถ้าอยากให้ไม่หายควรทำอย่างไร

ข้อมูลใน RAM/Memory จะหายไปทันทีเมื่อสั่ง Stop แอปพลิเคชันหรือเมื่อ Server ปิดลง

ถ้าอยากให้ไม่หายต้องทำการเชื่อมต่อและบันทึกข้อมูลลงในระบบจัดเก็บข้อมูลแบบถาวร เช่น Relational Database (MySQL, PostgreSQL) โดยใช้เทคโนโลยีอย่าง Spring Data JPA / Hibernate เข้ามาช่วยจัดการเชื่อมต่อฐานข้อมูล

4. Annotation แต่ละตัวทำหน้าที่อะไร

@RestController บอก Spring ว่าคลาสนี้คือ REST Controller ที่จะรับ HTTP Request และแปลงผลลัพธ์ของ Method (เช่น Object/List) ส่งกลับไปให้ Client เป็นรูปแบบ JSON อัตโนมัติ

@GetMapping ใช้แมป HTTP GET Request เข้ากับ Method ที่กำหนด สำหรับดึงข้อมูล

@PostMapping ใช้แมป HTTP POST Request เข้ากับ Method ที่กำหนด สำหรับสร้าง/เพิ่มข้อมูล

@PathVariable ใช้ดึงค่าจากตัวแปรบน URL Path (เช่น {id} จาก /coffees/{id}) มาใส่ในพารามิเตอร์ของ Method

@RequestBody ใช้แปลงข้อมูลที่ส่งมาในรูปแบบ JSON Payload จาก HTTP Request Body ให้กลายเป็น Java Object เพื่อนำไปใช้งานใน Method